

Frameworks of Modern AI

Module 2 — Programming for AI & Data Science

Get to know Lana — Fun Fact

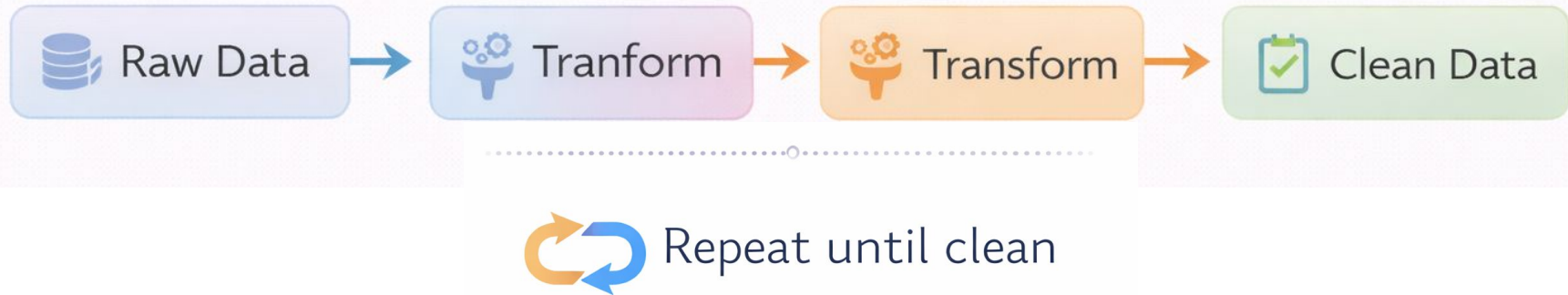


My Running Hobby

- 9 full marathons
- Emotion needs motion
- Yoga, weights, walking
- Goal: 10th full marathon by July 2028

Data Pipelines

Reliable Data Processing



Two Coding Approaches

Monolithic Script

```
import pandas as pd
import matplotlib as plt

df = pd.read_csv("sales.csv")

df["quantity"] = df["quantity"].fillna(
    df["quantity"].median())

df["revenue"] = df["quantity"] * df["unit_price"]

summary = df.groupby("category")["revenue"].sum()
plt.bar(summary.index, summary.values)
plt.savefig("chart.png")
```

Modular Pipeline

```
1 def load_data(path):
2     return pd.read_csv(path)

2 def clean_data(df):
    df["quantity"] = df["quantity"].
        fillna(df["quantity"].median())
3     return df

4 def add_features(df):
    df["revenue"] = df["quantity"] * df["unit_price"]

5 def summarize(df):
    return df.groupby("category")["revenue"].sum()

6 def visualize(summary):
    plt.bar(summary.index, summary.values)
    plt.savefig("chart.png")

if __name__ == "__main__":
1     df = load_data("sales.csv")
2     df = clean_data(df)
3     df = add_features(df)
4     summary = summarize(df)
5     visualize(summary)
```

Two Coding Approaches

```
monolithic.py:
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("sales.csv")
df["quantity"] = df["quantity"].fillna(df["quantity"].median())
df["revenue"] = df["quantity"] * df["unit_price"]
summary = df.groupby("category")["revenue"].sum()
plt.bar(summary.index, summary.values)
plt.savefig("chart.png")
```

```
modular.py:
import pandas as pd
import matplotlib.pyplot as plt

def load_data(path):
    return pd.read_csv(path)

def clean_data(df):
    df["quantity"] = df["quantity"].fillna(df["quantity"].median())
    return df

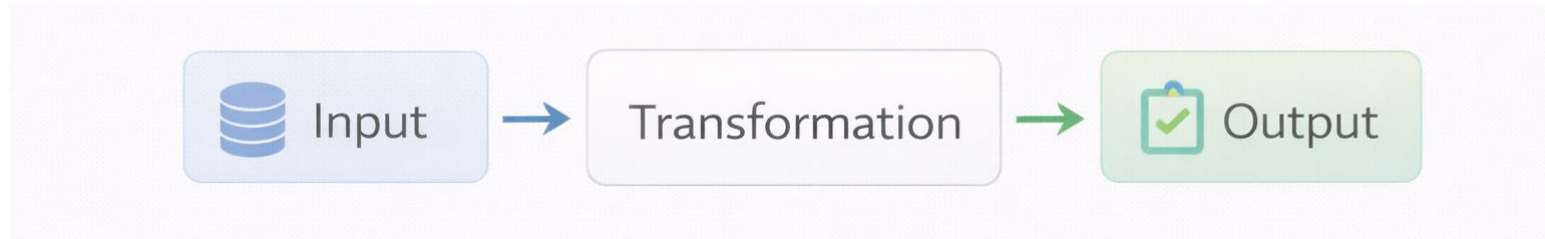
def add_features(df):
    df["revenue"] = df["quantity"] * df["unit_price"]
    return df

def summarize(df):
    return df.groupby("category")["revenue"].sum()

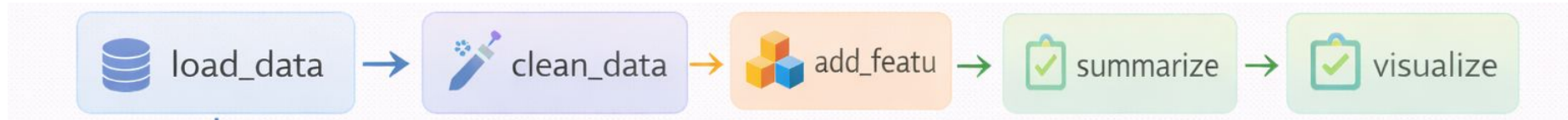
def visualize(summary):
    plt.bar(summary.index, summary.values)
    plt.savefig("chart.png")

if __name__ == "__main__":
    df = load_data("sales.csv")
    df = clean_data(df)
    df = add_features(df)
    summary = summarize(df)
    visualize(summary)
```

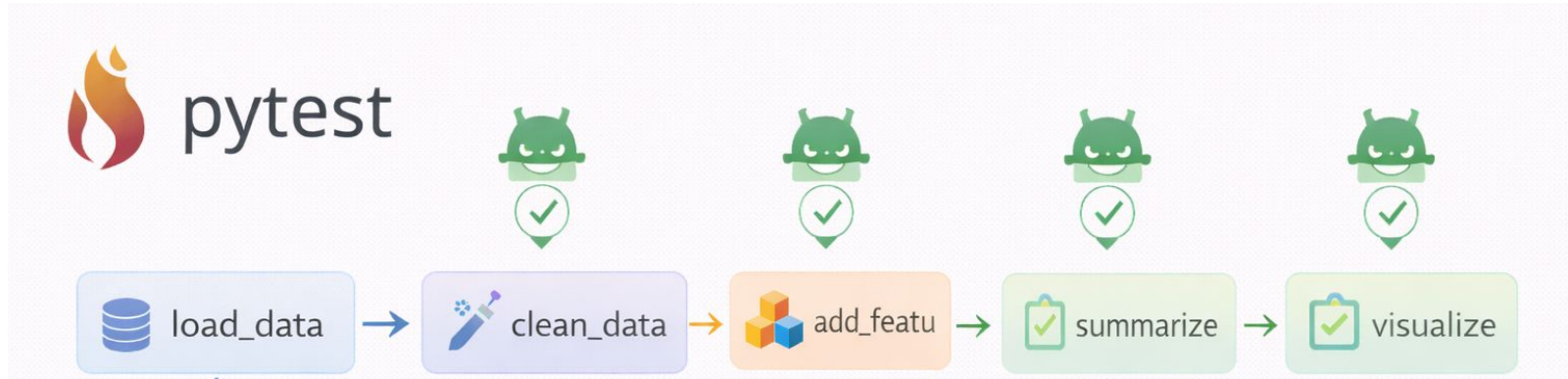
Pipeline Contracts



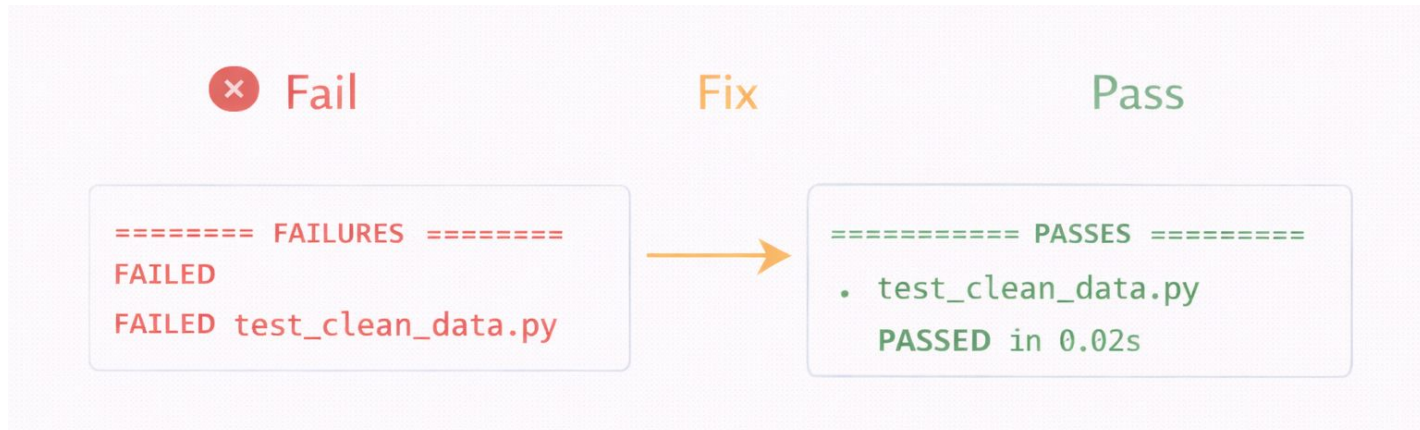
Example Pipeline



Testing Pipelines

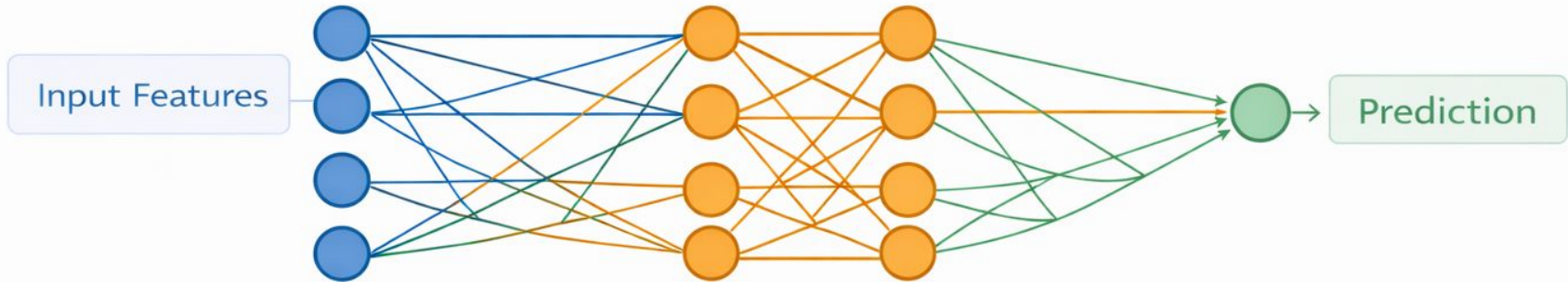


Red Green Cycle

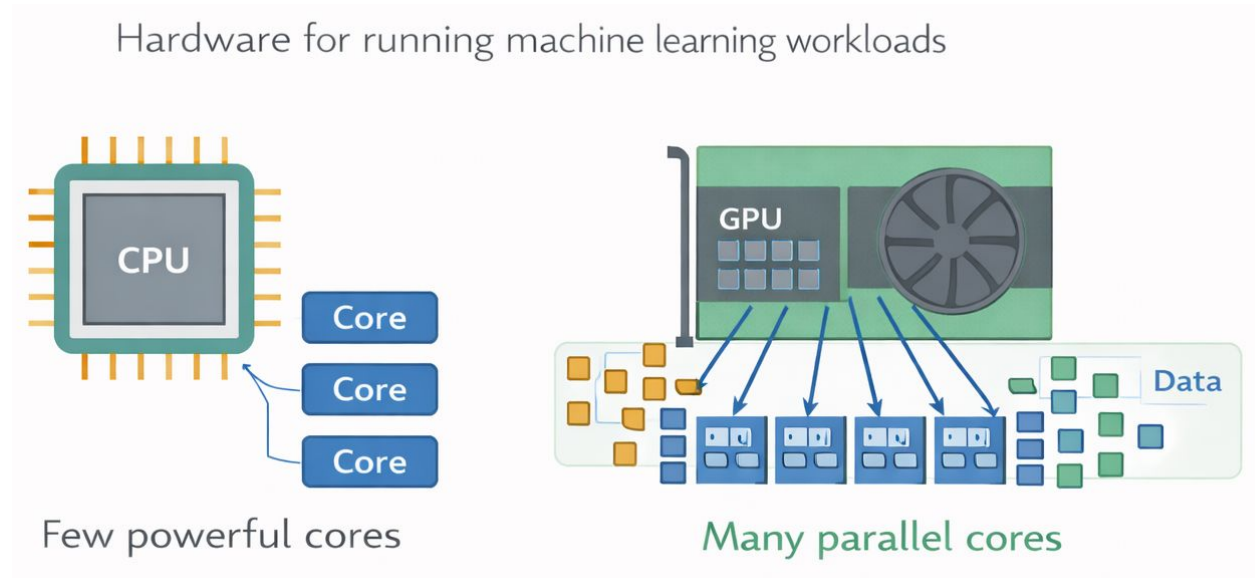
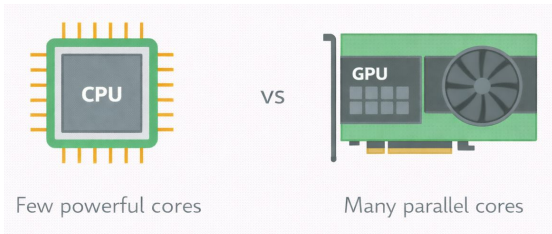


What is Deep Learning?

Neural networks that learn patterns from data



Hardware Decisions: CPUs and GPUs

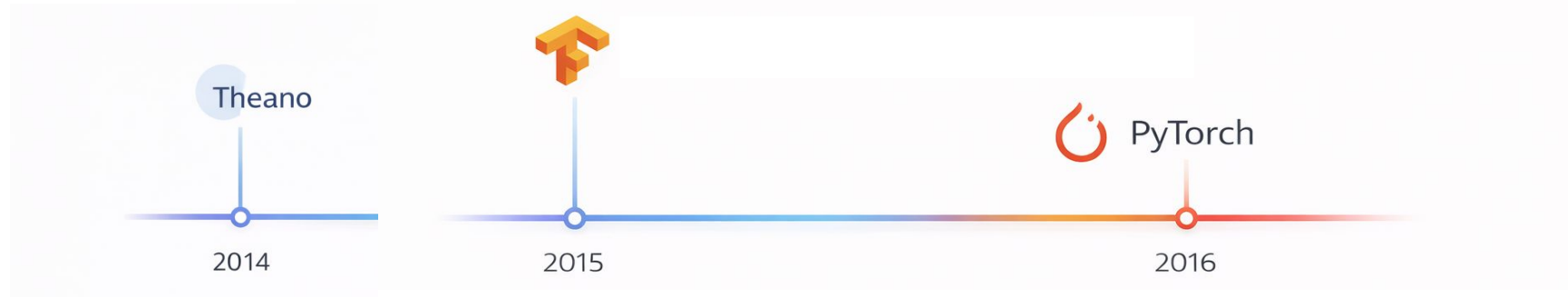


AI.SPIRE

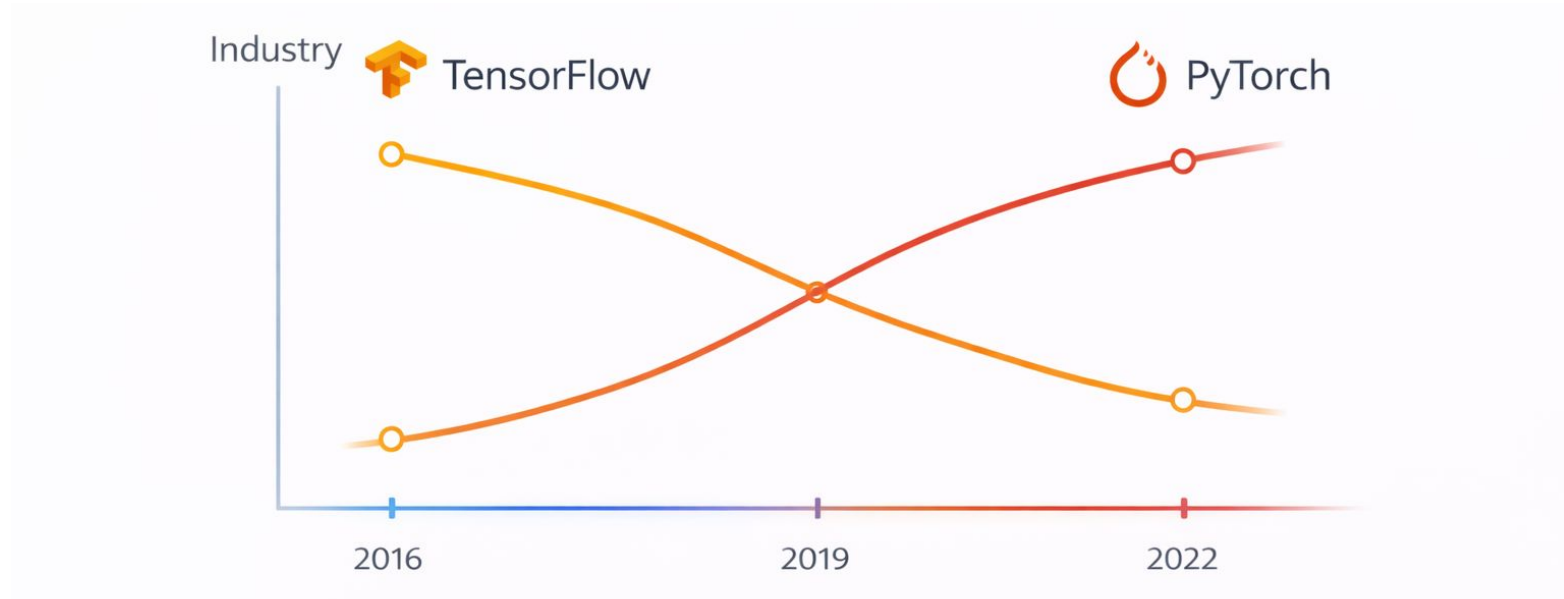
Deep Learning Framework Landscape

Framework Origins

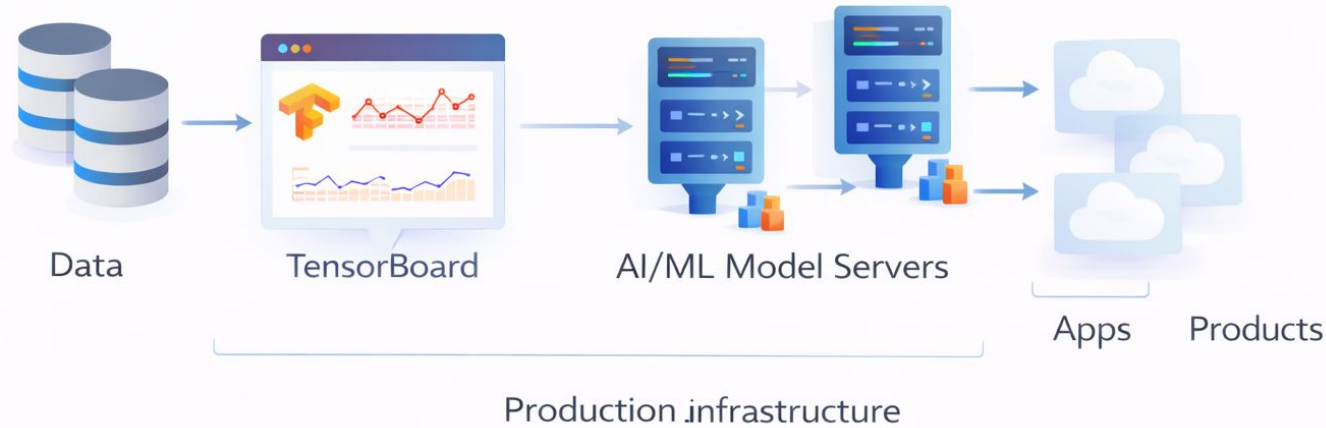
- TensorFlow – Google Brain (2015)
- PyTorch – Facebook AI Research (2016)



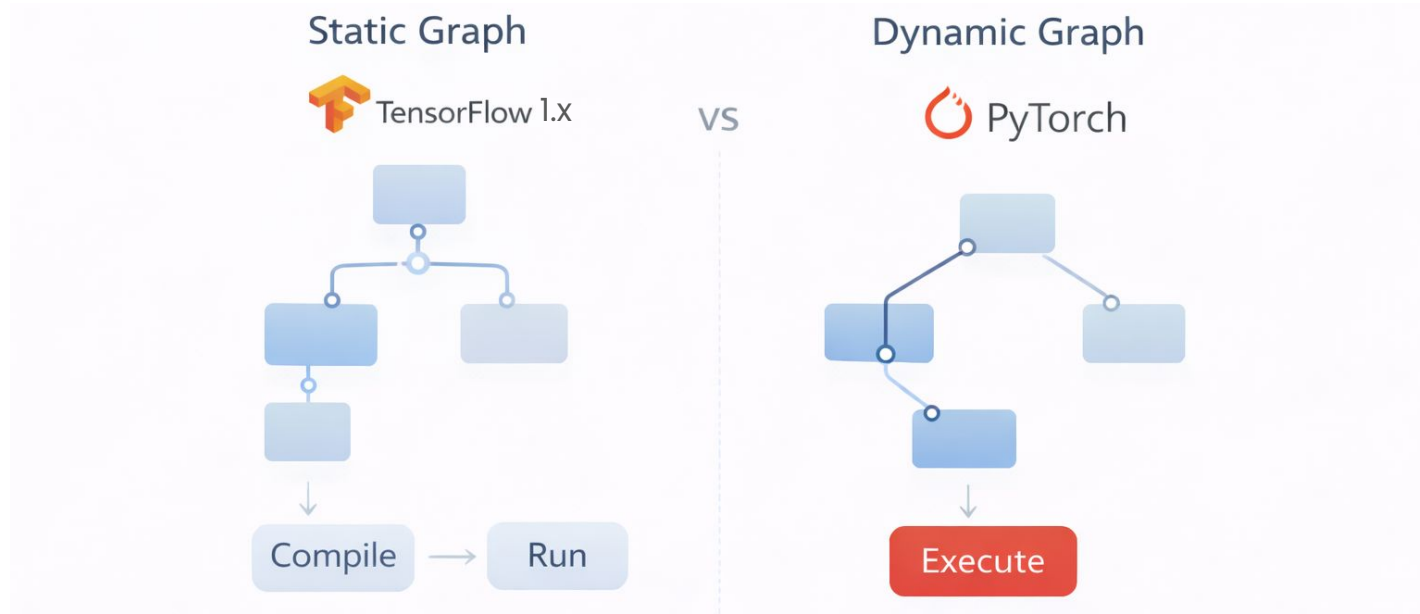
Adoption Over Time



Why would business adopt TensorFlow first?



The Architectural Difference

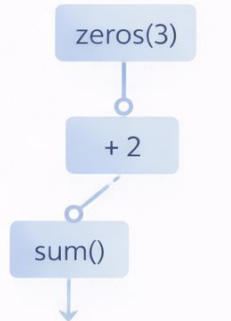


Dynamic Graph Execution

- Define by run



```
import torch  
x = torch.zeros(3)  
y = x + 2  
z = torch.sum(y)
```

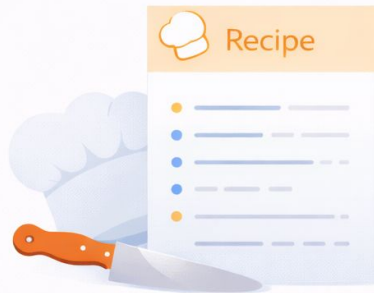


Built dynamically
while running

A Simple Analogy

Recipe vs Cooking

Recipe



TensorFlow 1.x

Cooking



PyTorch



TensorFlow 2.x Caught Up To PyTorch

TensorFlow 1.x	TensorFlow 2.x
– Static graph definition – Session-based execution – Focus on model serving	⚡ Eager execution 🏗️ @tf.function optimization 🔧 Focus on code development

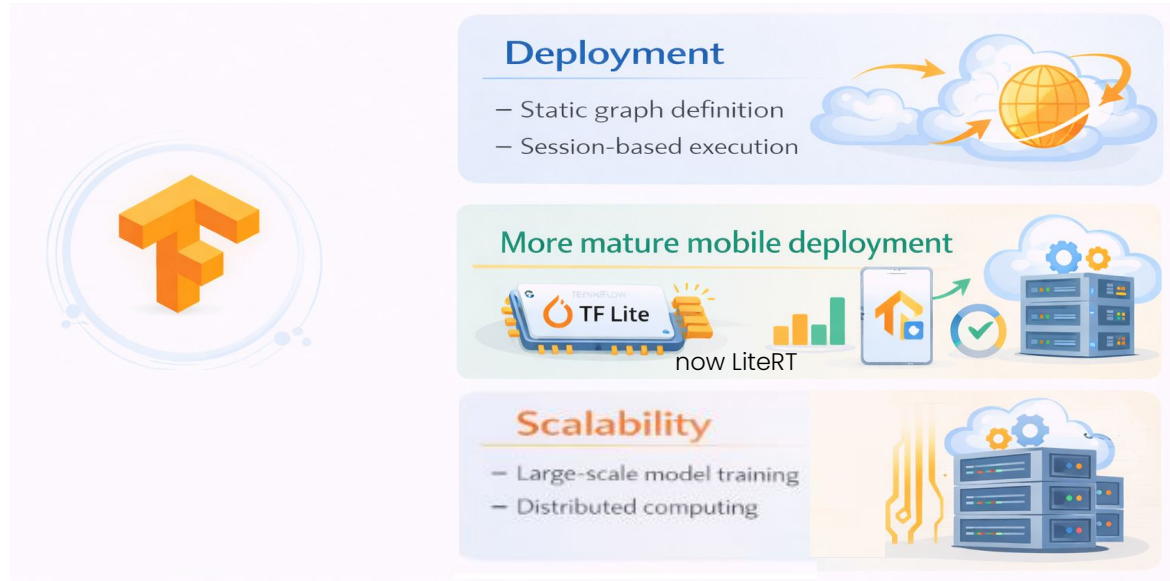
Eager Execution

- Lines run immediately
⚡ → Like PyTorch
- Step-by-step debugging 🔍

@tf.function

- 🏗️ Graph Compilation
- 🕒 High Performance

TensorFlow Strengths



PyTorch Strengths



Hugging Face



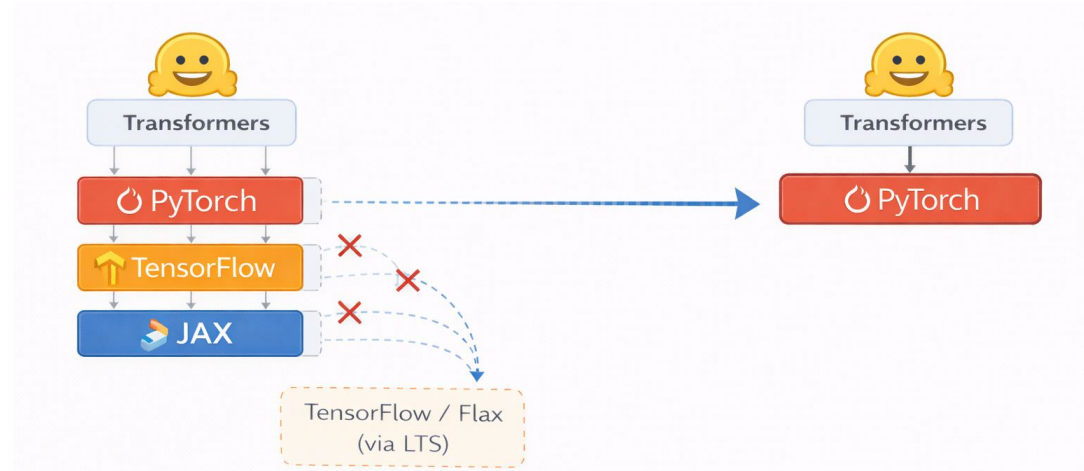
Hugging Face Transformers ↔

[NLP]

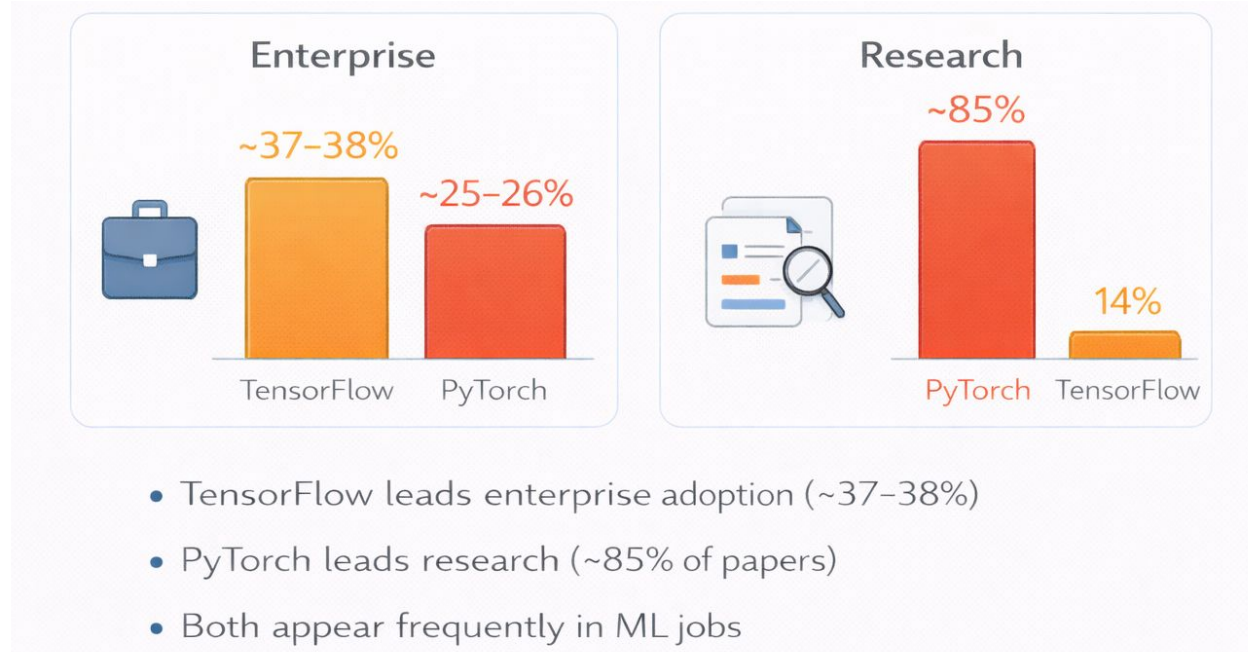
[Computer Vision]

[Speech]

[Generative AI]

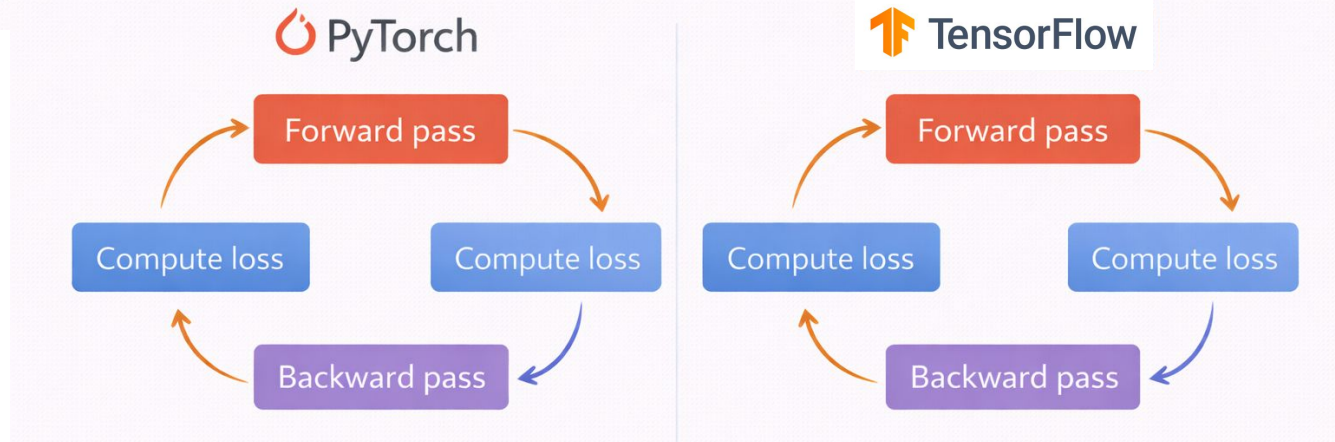


Real Usage Today

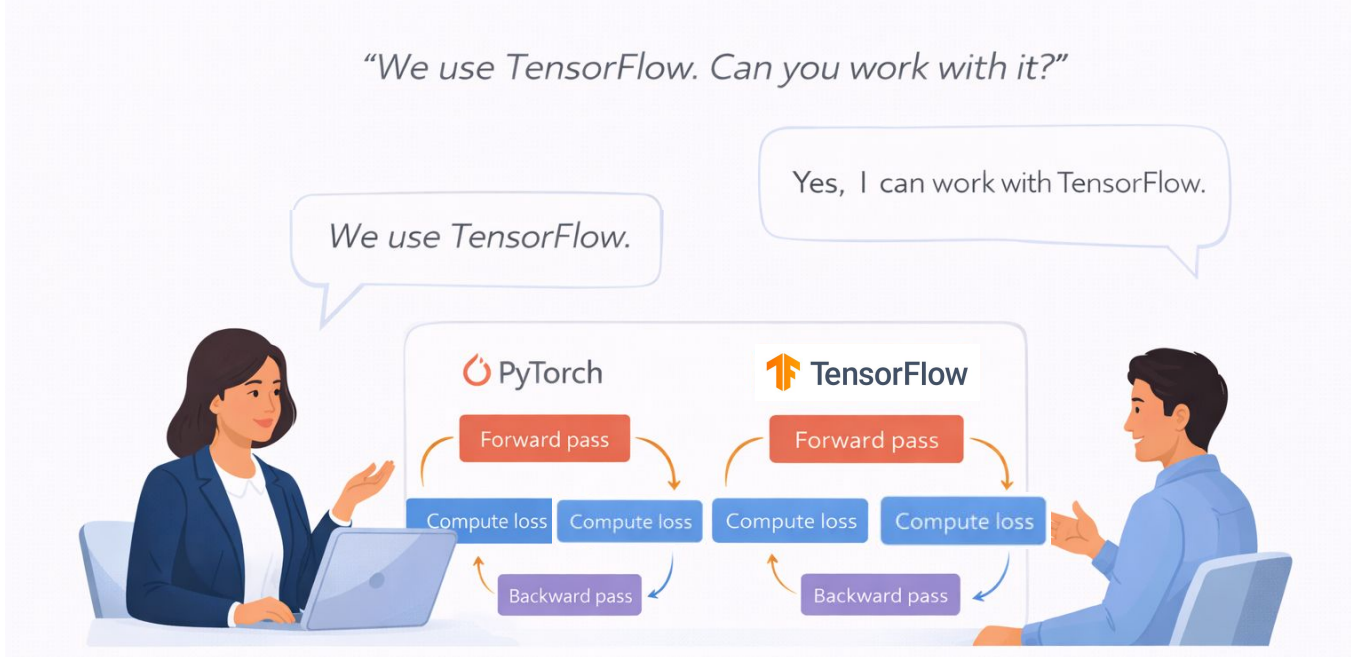


Why This Course Uses PyTorch

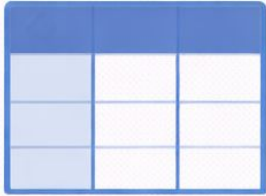
- Dynamic
- Readable
- Industry r
- Architecture transfers
- Syntax changes



Interview Scenario



Data Containers



DataFrame

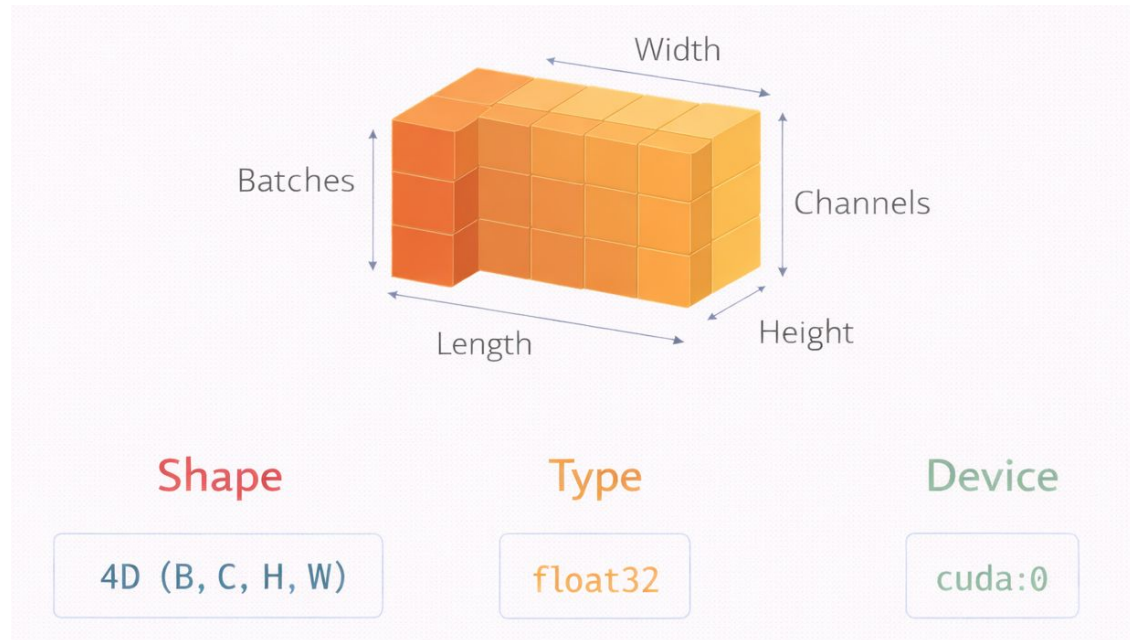


NumPy



Tensor

Tensor Properties



DataFrame to Tensor in Three Lines

```
import pandas as pd
import torch
df = pd.DataFrame([1, 4, 6], [2, 5, 7], [3, 6, 8]))
tensor = torch.tensor(df.values)
```

DataFrame

feature_1	feature_2	feature_3
1	4	6
2	5	7
3	6	8



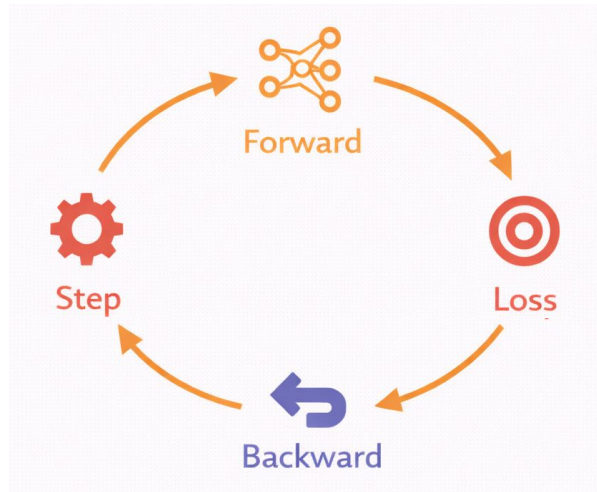
Tensor

[1, 4, 6]
[2, 5, 7]
[3, 6, 8]

AI.SPIRE

The Training Loop

The Model Training Loop



Forward: model predicts

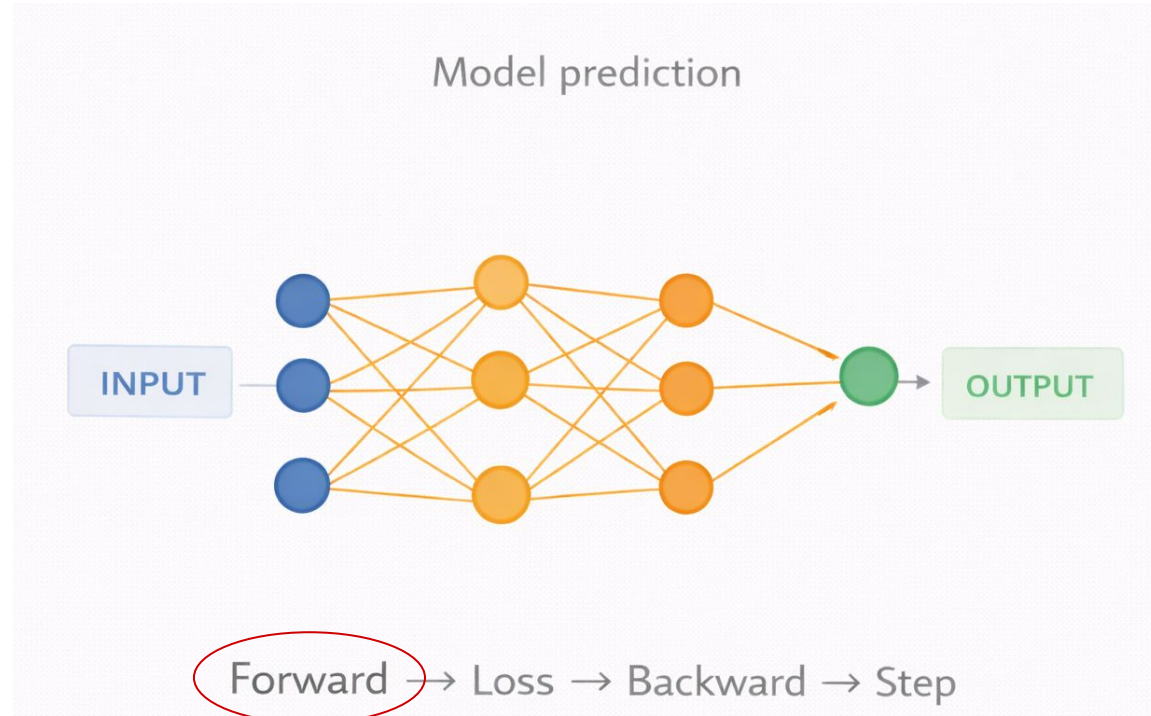
Loss: measure error

Backward: compute gradients

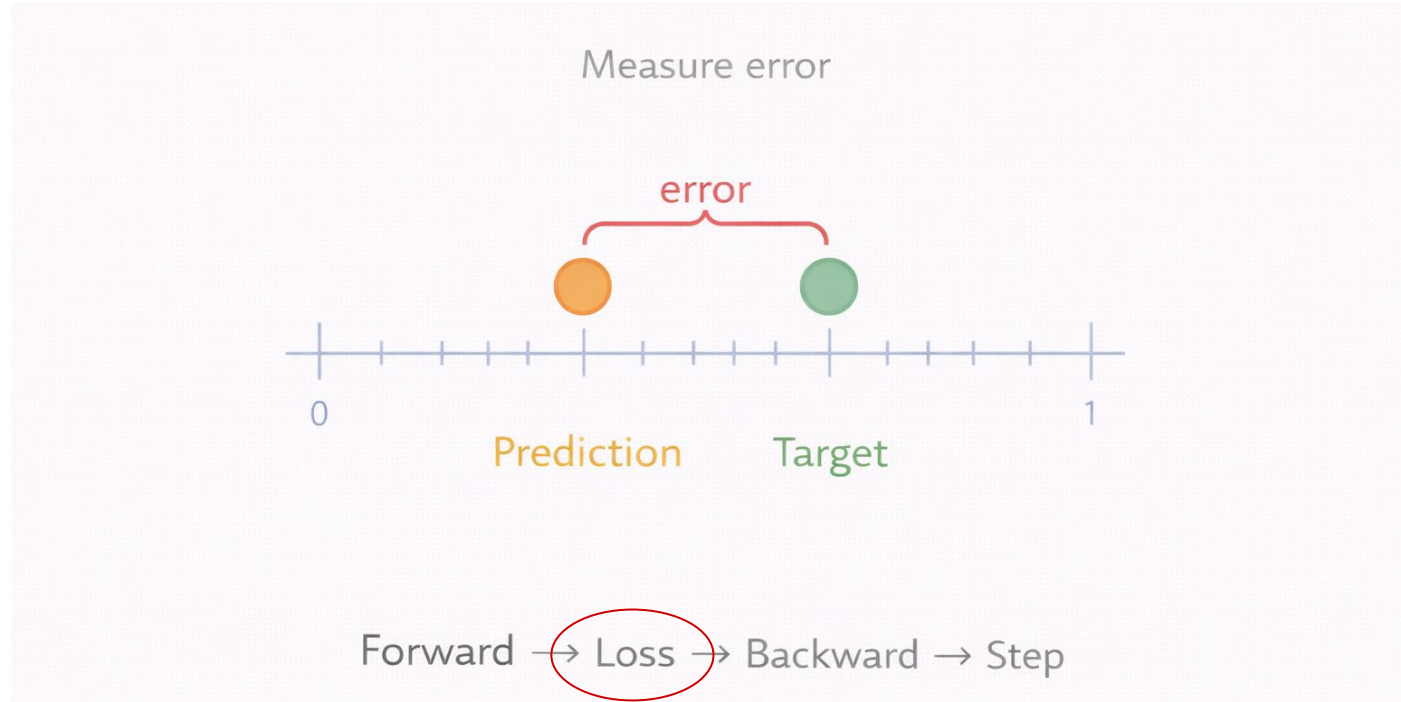
Step: update weights

Repeat. Each time, the loss gets smaller.

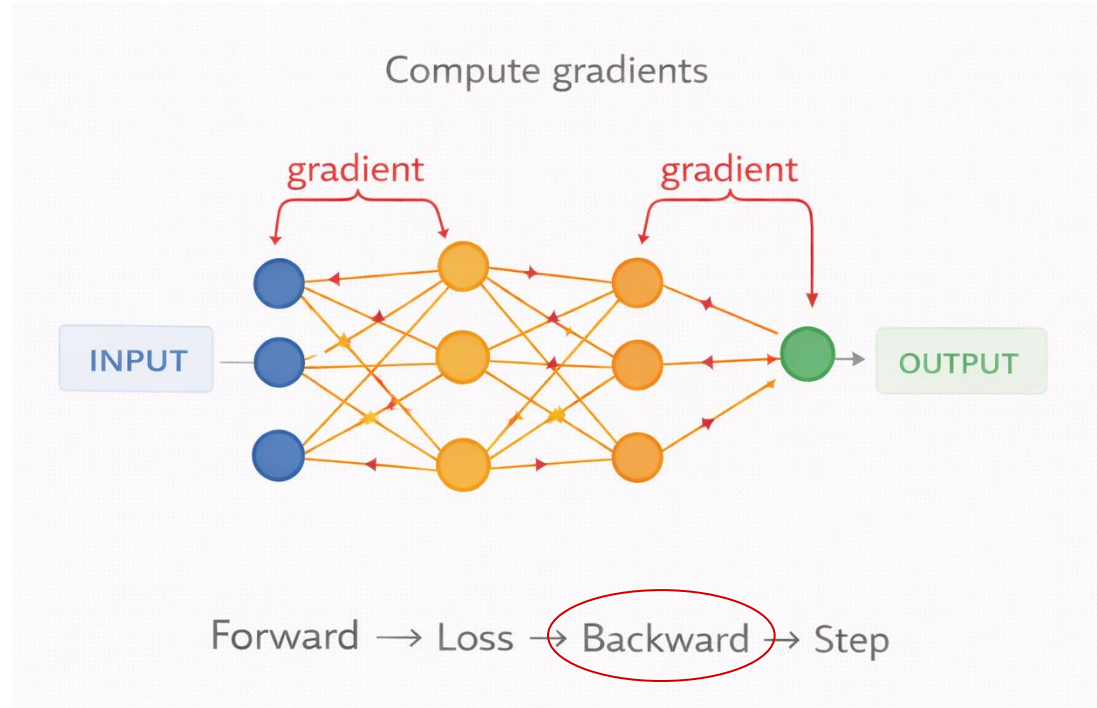
Forward



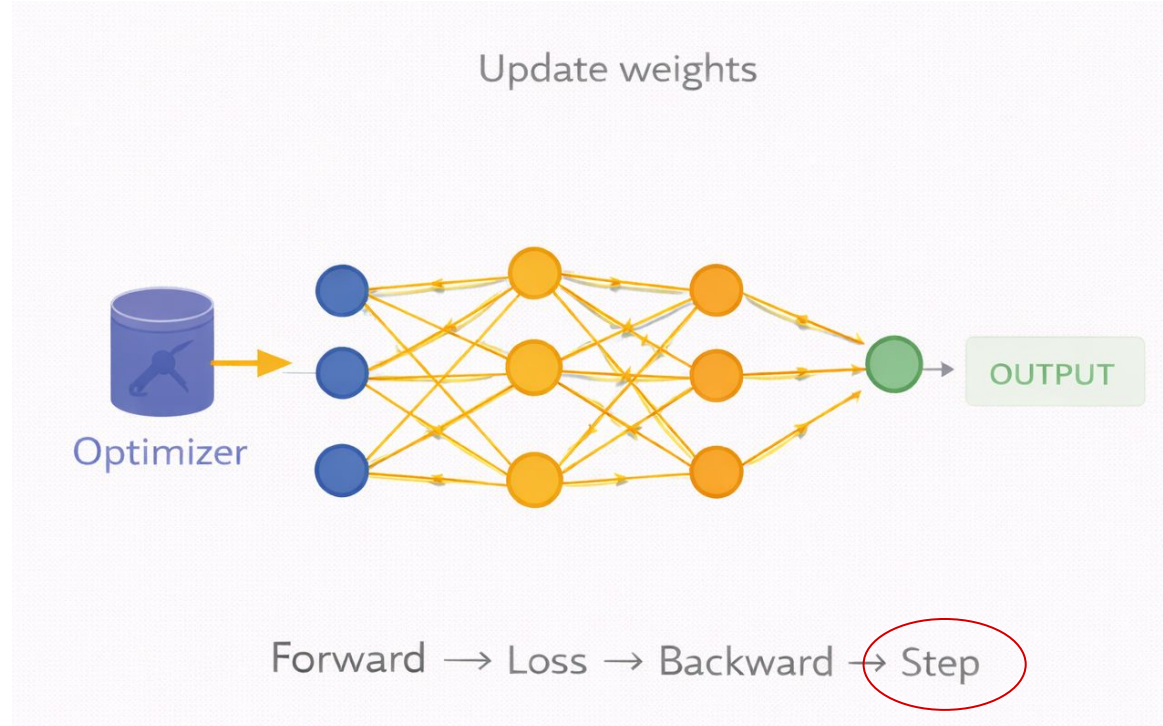
Loss



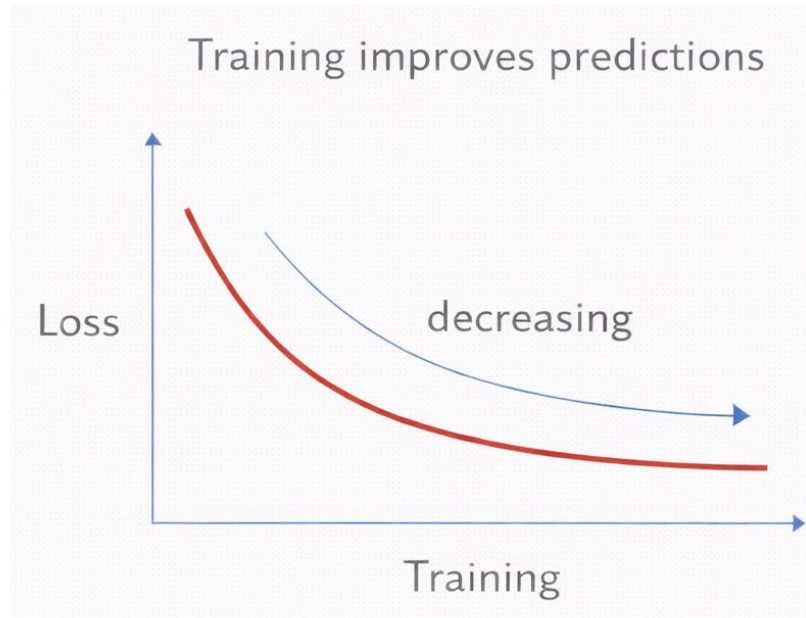
Backward



Step



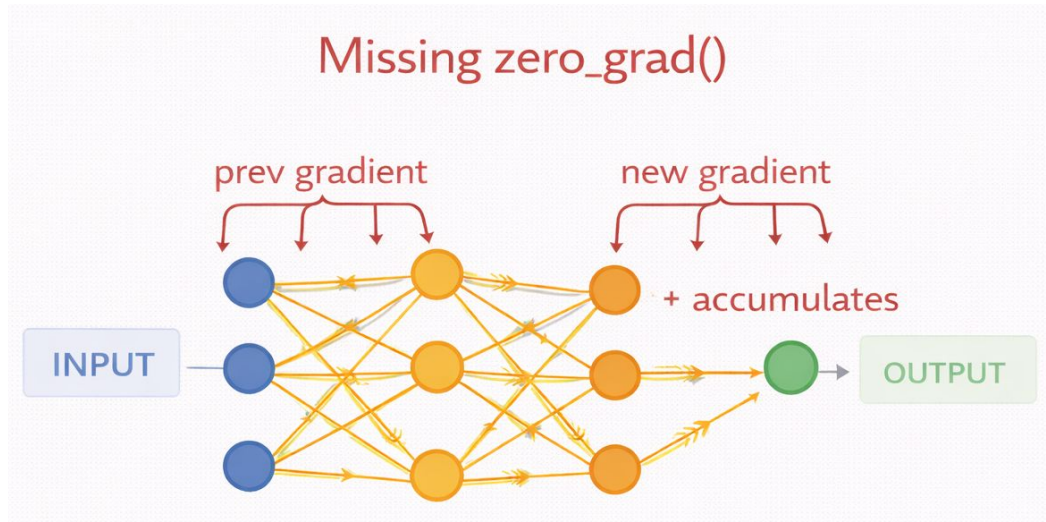
Repeat



One epoch:

Forward → Loss → Backward → Step

Common Error



Gradients accumulate by default.

Previous + current = wrong update.

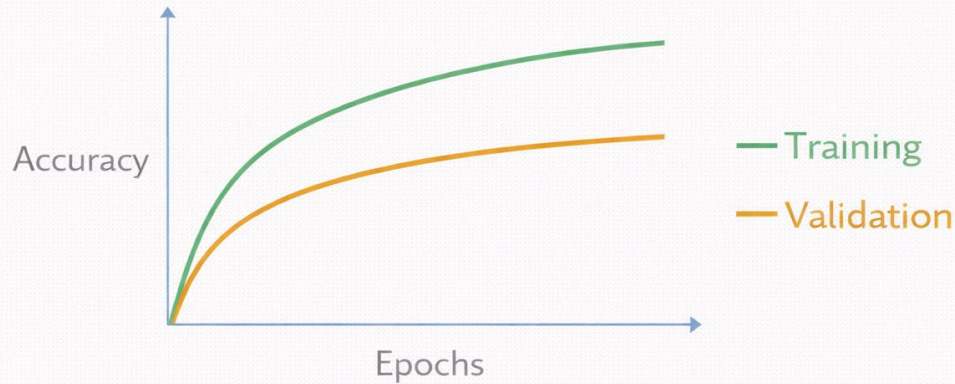
Always zero gradients before backward:

```
optimizer.zero_grad()
```

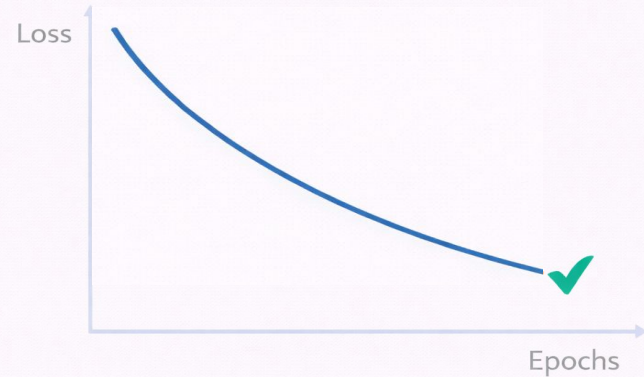
Before the forward pass.

Model Evaluation

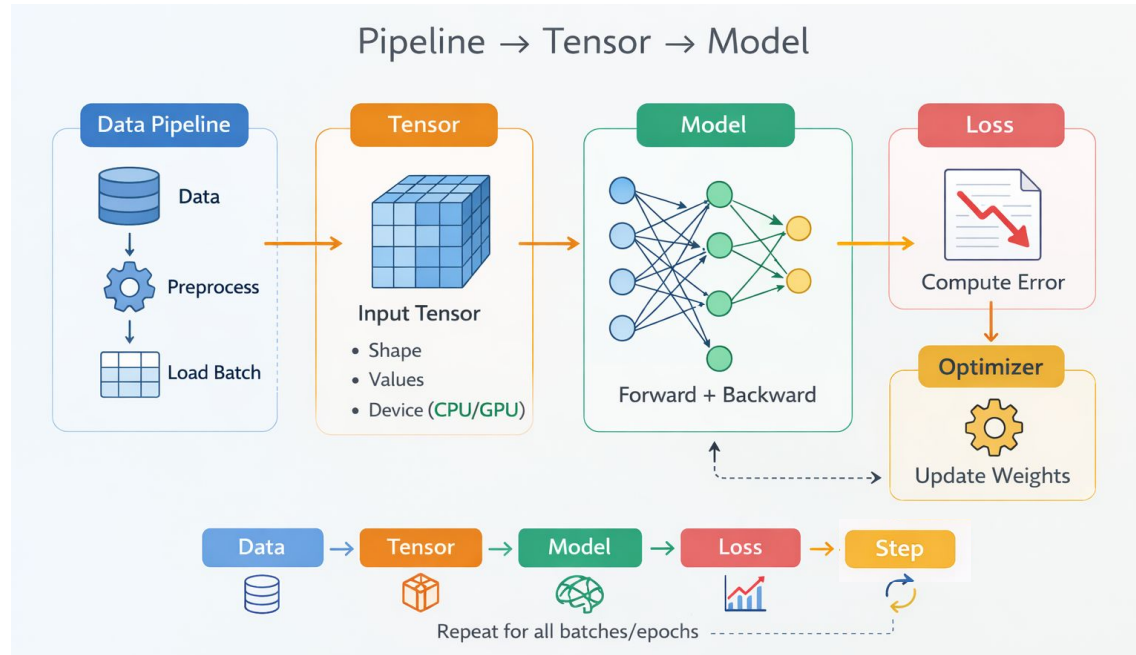
Loss $\neq$ performance



For now: loss decreasing means training is working.



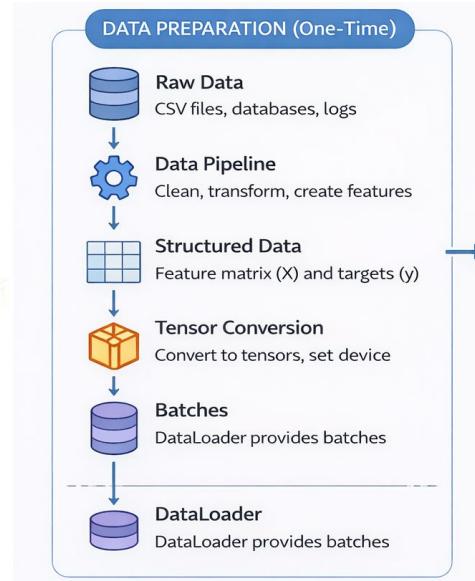
Full Workflow



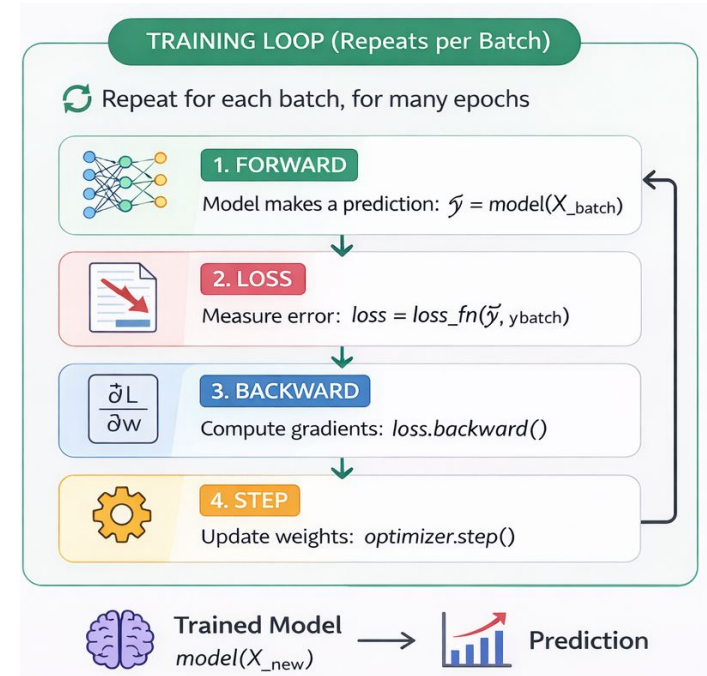
What Happens Next

```
drill_functions.py | test_drill.py
1 def clean_column(series):
2   median = series.median()
3   return series.fillna(median)
4
5 def compute_revenue(quantity, price):
6   return quantity * price
7
8
9 test_drill.py
> pytest test_drill.py
===== session starts =====
platform darwin -- Python 3...
rootdir: ...
collected 2 items
✓ test_drill.py::test_clean_column PASSED
✓ test_drill.py::test_compute_revenue PASSED
===== Start of each iteration =====
```

Drill



Lab



Integration

Questions?